CHAPTER OUTLINE

6.1 What Is an Index?

6.2 A Simple Index with an Entry-Sequenced File

6.3 Basic Operations on an Indexed, Entry-Sequenced File

6.4 Indexes That Are Too Large to Hold in Memory

6.5 Indexing to Provide Access by Multiple Keys

6.6 Retrieval Using Combinations of Secondary Keys

6.7 Improving the Secondary Index Structure: Inverted Lists

6.7.1 A First Attempt at a Solution

6.7.2 A Better Solution: Linking the List of References

6.8 Selective Indexes

6.9 Binding

6.1 What Is an Index?

The last few pages of many books contain an index. Such an index is a table containing a list of topics (keys) and numbers of pages where the topics can be found (reference fields).

All indexes are based on the same basic concept—keys and reference fields. The types of indexes we examine in this chapter are called *simple indexes* because they are represented using *simple arrays* of structures that contain the keys and reference fields. In later chapters we look at indexing schemes that use more complex data structures, especially trees. In this chapter, however, we want to emphasize that indexes can be very simple and still provide powerful tools for file processing.

The index to a book provides a way to find a topic quickly. If you have ever had to use a book without a good index, you already know that an index is a desirable alternative to scanning through the book sequentially to find a topic. In general, indexing is another way to handle the problem that we explored in Chapter 5: An index is a way to find things.

Consider what would happen if we tried to apply the previous chapter's methods, sorting and binary searching, to the problem of finding things in a book. Rearranging all the words in the book so they were in alphabetical order certainly would make finding any particular term easier but would obviously have disastrous effects on the meaning of the book. In a sense, the terms in the book are pinned records. This is an absurd example, but it clearly underscores the power and importance of the index as a conceptual tool. Since it works by indirection, *an index lets you impose order on a file without actually rearranging the file*. This not only keeps us from disturbing

pinned records, but also makes matters such as record addition much less expensive than they are with a sorted file.

Take, as another example, the problem of finding books in a library. We want to be able to locate books by a specific author, by their titles, or by subject areas. One way of achieving this is to have three copies of each book and three separate library buildings. All of the books in one building would be sorted by author's name, another building would contain books arranged by title, and the third would have them ordered by subject. Again, this is an absurd example, but one that underscores another important advantage of indexing. Instead of using multiple arrangements, a library uses a card catalog. The card catalog is actually a set of three indexes, each using a different *key field*, and all of them using the same catalog number as a *reference field*. Another use of indexing, then, is to provide *multiple access paths* to a file.

We also find that indexing gives us *keyed access to variable-length record files*. Let's begin our discussion of indexing by exploring this problem of access to variable-length records and the simple solution that indexing provides.

6.2 A Simple Index with an Entry-Sequenced File

Suppose we own an extensive collection of musical recordings and we want to keep track of the collection through the use of computer files. For each recording, we keep the information shown in Fig. 6.1. The data file records are variable length. Figure 6.2 illustrates such a collection of data records. We refer to this data record file as *Datafile*.

There are a number of approaches that could be used to create a variable-length record file to hold these records; the record addresses used in Fig. 6.2 suggest that each record be preceded by a size field that permits skip sequential access and easier file maintenance. This is the structure we use.

Suppose we formed a *primary key* for these records consisting of the initials for the record company label combined with the record company's

Identification number
Title
Composer or composers
Artist or artists
Label (publisher)

FIGURE 6.1 Contents of a data record.

Rec. addr.	Label	ID number	Title	Composer(s)	Artist(s)
32†	LON	2312	Romeo and Juliet	Prokofiev	Maazel
77	RCA	2626	Quartet in C Sharp Minor	Beethoven	Julliard
132	WAR	23699	Touchstone	Corea	Corea
167	ANG	3795	Symphony No. 9	Beethoven	Giulini
211	COL	38358	Nebraska	Springsteen	Springsteen
256	DG	18807	Symphony No. 9	Beethoven	Karajan
300	MER	75016	Coq d'or Suite	Rimsky-Korsakov	Leinsdorf
353	COL	31809	Symphony No. 9	Dvorak	Bernstein
396	DG	139201	Violin Concerto	Beethoven	Ferras
442	FF	245	Good News	Sweet Honey in the Rock	Sweet Honey in the Rock

†Assume there is a header record that uses the first 32 bytes.

FIGURE 6.2 Sample contents of *Datafile*.

ID number. This will make a good primary key since it should provide a *unique* key for each entry in the file. We call this key the *Label ID*. The canonical form for the Label ID consists of the uppercase form of the Label field followed immediately by the ASCII representation of the ID number. For example,

```
LON2312
```

How could we organize the file to provide rapid keyed access to individual records? Could we sort the file and then use binary searching? Unfortunately, binary searching depends on being able to jump to the middle record in the file. This is not possible in a variable-length record file because direct access by relative record number is not possible—there is no way to know where the middle record is in any group of records.

An alternative to sorting is to construct an index for the file. Figure 6.3 illustrates such an index. On the right is the data file containing information about our collection of recordings, with one variable-length data record per recording. Only four fields are shown (Label, ID number, Title, and Composer), but it is easy to imagine the other information filling out each record.

On the left is the index file, each record of which contains a 12-character *key* (left justified, blank filled) corresponding to a certain Label ID in the data file. Each key is associated with a *reference field* giving the address of the

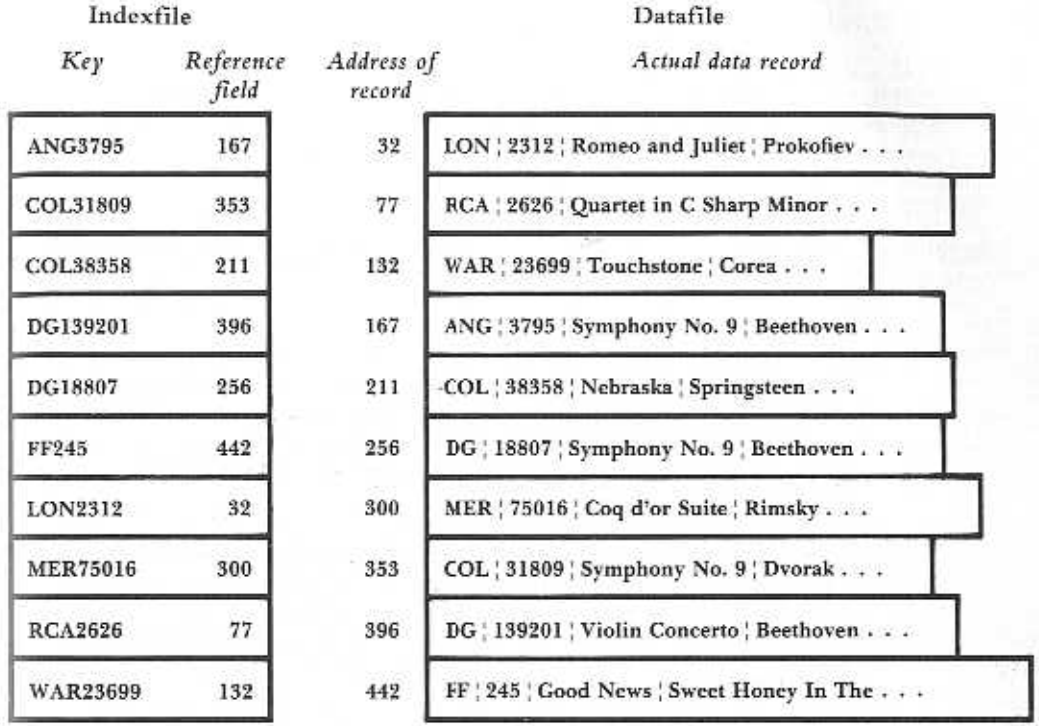


FIGURE 6.3 Sample index with corresponding data file.

first byte of the corresponding data record. ANG3795, for example, corresponds to the reference field containing the number 167, meaning that the record containing full information on the recording with Label ID ANG3795 can be found starting at byte number 167 in the record file.

The structure of the index file is very simple. It is a fixed-length record file in which each record has two fixed-length fields: a key field and a byte-offset field. There is one record in the index file for every record in the data file.

Note also that the index is sorted, whereas the data file is not. Consequently, although Label ID ANG3795 is the first entry in the index; it is not necessarily the first entry in the data file. In fact, the data file is *entry sequenced*, which means that the records occur in the order that they are entered into the file. As we see soon, the use of an entry-sequenced file can make record addition and file maintenance much simpler than is the case with a data file that is kept sorted by some key.

```
PROCEDURE retrieve_record(KEY)

    find position of KEY in Indexfile  /* Probably using binary search */
    look up the BYTE_OFFSET of the corresponding record in Datafile
    use SEEK() and the byte_offset to move to the data record
    read the record from Datafile

end PROCEDURE
```

FIGURE 6.4 *Retrieve_record():* a procedure to retrieve a single record from *Datafile* through *Indexfile*.

Using the index to provide access to the data file by Label ID is a simple matter. The steps needed to retrieve a single record with key KEY from *Datafile* are shown in the procedure *retrieve_record( )* in Fig. 6.4. Although this retrieval strategy is relatively straightforward, it contains some features that deserve comment:

- We are now dealing with two files—the index file and the data file. The index file is considerably easier to work with than the data file because it uses fixed-length records (which is why we can search it with a binary search) and because it is likely to be much smaller than the data file.
- By requiring that the index file have fixed-length records, we impose a limit on the sizes of our keys. In this example we assume that the primary key field is long enough to retain every key's unique identity. The use of a small, fixed key field in the index could cause problems if a key's uniqueness is truncated away as it is placed in the fixed index field.
- In the example, the index carries no information other than the keys and the reference fields, but this need not be the case. We could, for example, keep the length of each *Datafile* record in *Indexfile*.

6.3 Basic Operations on an Indexed, Entry-Sequenced File

We have noted that the process of keeping files sorted to permit binary searching for records can be very expensive. One of the great advantages of using a simple index with an entry-sequenced data file is that record addition can take place much more quickly than with a sorted data file as long as the index is small enough to be held entirely in memory. If the index record length is short, this is not a difficult condition to meet for small files consisting of no more than a few thousand records. For the moment our

discussions assume that the condition is met and that the index is read from secondary storage into an array of structures called INDEX[ ]. Later we consider what should be done when the index is too large to fit into memory.

Keeping the index in memory as the program runs also lets us find records by key more quickly with an indexed file than with a sorted one since the binary searching can be performed entirely in memory. Once the byte offset for the data record is found, then a single seek is all that is required to retrieve the record. The use of a sorted data file, on the other hand, requires a seek for each step of the binary search.

The support and maintenance of an entry-sequenced file coupled with a simple index requires the development of procedures to handle a number of different tasks. Besides the *retrieve_record( )* algorithm described previously, other procedures used to find things by means of the index include the following:

- Create the original empty index and data files;
- Load the index file into memory before using it;
- Rewrite the index file from memory after using it;
- Add records to the data file and index;
- Delete records from the data file; and
- Update records in the data file.

Creating the Files Both the index file and the data file are created as empty files, with header records and nothing else. This can be accomplished quite easily by creating the files and writing headers to both files.

Loading the Index into Memory We assume that the index file is small enough to fit into primary memory, so we define an array *INDEX[ ]* to hold the index records. Each array element has the structure of an index record. Loading the index file into memory, then, is simply a matter of reading in and saving the index header record and then reading the records from the index file into the INDEX[ ] array. Since this will be a sequential read, and since the records are short, the procedure should be written so it reads a large number of index records at once, rather than one record at a time.

Rewriting the Index File from Memory When processing of an indexed file is completed, it is necessary to rewrite INDEX[ ] back into the index file if the array has been changed in any way. In Fig. 6.5, the procedure *rewrite_index( )* describes the steps for doing this.

```
PROCEDURE rewrite_index()

    check a status flag that tells whether the INDEX[] array
        has been changed in any way.

    if there were changes, then
        open the index file as a new empty file
        update the header record and rewrite the header
        write the index out to the newly created file

    close the index file

end PROCEDURE
```

FIGURE 6.5 The *rewrite_index()* procedure.

It is important to consider what happens if this rewriting of the index does not take place, or takes place incompletely. Programs do not always run to completion. A program designer needs to guard against power failures, against the operator turning the machine off at the wrong time, and other such disasters. One of the serious dangers associated with reading an index into memory and then writing it out when the program is over is that the copy of the index on disk will be out of date and incorrect if the program is interrupted. It is imperative that a program contain at least the following two safeguards to protect against this kind of error:

- There should be a mechanism that permits the program to know when the index is out of date. One possibility involves setting a status flag as soon as the copy of the index in memory is changed. This status flag could be written into the header record of the index file on disk as soon as the index is read into memory, and then subsequently cleared when the index is rewritten. All programs could check the status flag before using an index. If the flag is found to be set, then the program would know that the index is out of date.
- If a program detects that an index is out of date, the program must have access to a procedure that reconstructs the index from the data file. This should happen automatically, taking place before any attempt is made to use the index.

Record Addition Adding a new record to the data file requires that we also add a record to the index file. Adding to the data file itself is easy. The exact procedure depends, of course, on the kind of variable-length file organization being used. In any case, when we add a data record we should know the starting *byte_offset* of the file location at which we wrote the record. This information, along with the canonical form of the record's key, must be placed in the INDEX[ ] array.

Since the INDEX[ ] array is kept in sorted order by key, insertion of the new index record probably requires some rearrangement of the index. In a way, the situation is similar to the one we face as we add records to a sorted data file. We have to shift or slide all the records that have keys that come in order after the key of the record we are inserting. The shifting opens up a space for the new record. The big difference between the work we have to do on the index records and the work required for a sorted data file is that the INDEX[ ] array is contained *wholly in memory*. All of the index rearrangement can be done without any file access.

Record Deletion In Chapter 5 we describe a number of approaches to deleting records in variable-length record files that allow for the reuse of the space occupied by these records. These approaches are completely viable for our data file since, unlike a sorted data file, the records in this file need not be moved around to maintain an ordering on the file. This is one of the great advantages of an indexed file organization: We have rapid access to individual records by key without disturbing pinned records. In fact, the indexing itself pins all the records.

Of course, when we delete a record from the data file we must also delete the corresponding entry from our index file. Since the index is contained in an array during program execution, deleting the index record and shifting the other records to close up the space may not be an overly expensive operation. Alternatively, we could simply mark the index record as deleted, just as we might mark the corresponding data record.

Record Updating Record updating falls into two categories:

- *The update changes the value of the key field.* This kind of update can bring about a reordering of the index file as well as the data file. Conceptually, the easiest way to think of this kind of change is as a deletion followed by an addition. This delete/add approach can be implemented while still providing the program user with the view that he or she is merely changing a record.
- *The update does not affect the key field.* This second kind of update does not require rearrangement of the index file, but may well involve reordering of the data file. If the record size is unchanged or decreased by the update, the record can be written directly into its old space, but if the record size is increased by the update, a new slot for the

record will have to be found. In the latter case the starting address of the rewritten record must replace the old address in the *byte_offset* field of the corresponding index record.

6.4 Indexes That Are Too Large to Hold in Memory

The methods we have been discussing, and, unfortunately, many of the advantages associated with them, are tied to the assumption that the index file is small enough to be loaded into memory in its entirety. If the index is too large for this approach to be practical, then index access and maintenance must be done on secondary storage. With simple indexes of the kind we have been discussing, accessing the index on a disk has the following disadvantages:

- Binary searching of the index requires several seeks rather than taking place at electronic memory speeds. Binary searching of an index on secondary storage is not substantially faster than the binary searching of a sorted file.
- Index rearrangement due to record addition or deletion requires shifting or sorting records on secondary storage. This is literally millions of times more expensive than the cost of these same operations when performed in electronic memory.

Although these problems are no worse than those associated with the use of any file that is sorted by key, they are severe enough to warrant the consideration of alternatives. Any time a simple index is too large to hold in memory, you should consider using

- A *hashed* organization if access speed is a top priority; or
- A *tree-structured* index, such as a *B-tree*, if you need the flexibility of both keyed access and ordered, sequential access.

These alternative file organizations are discussed at length in the chapters that follow. But, before writing off the use of simple indexes on secondary storage altogether, we should note that they provide some important advantages over the use of a data file sorted by key even if the index cannot be held in memory:

- A simple index makes it possible to use a binary search to obtain keyed access to a record in a variable-length record file. The index provides the service of associating a fixed-length and therefore binary-searchable record with each variable-length data record.

- If the index records are substantially smaller than the data file records, sorting and maintaining the index can be less expensive than would be sorting and maintaining the data file. This is simply because there is less information to move around in the index file.
- If there are pinned records in the data file, the use of an index lets us rearrange the keys without moving the data records.

There is another advantage associated with the use of simple indexes, one that we have not yet discussed. It, in itself, can be reason enough to use simple indexes even if they do not fit into memory. Remember the analogy between an index and a library card catalog? The card catalog provides multiple views or arrangements of the library's collection, even though there is only one set of books arranged in a single order. Similarly, we can use multiple indexes to provide multiple views of a data file.

6.5 Indexing to Provide Access by Multiple Keys

One question that might reasonably arise at this point is, "All this indexing business is pretty interesting, but who would ever want to find a record using a key such as DG18807? What I want is the Symphony No. 9 record by Beethoven."

Let's return to our analogy between our index and a library card catalog. Suppose we think of our primary key, the Label ID, as a kind of catalog number. Like the catalog number assigned to a book, we have taken care to make our Label ID unique. Now, in a library it is very unusual to begin by looking for a book with a particular catalog number (e.g., "I am looking for a book with a catalog number QA331T5 1959."). Instead, one generally begins by looking for a book on a particular subject, with a particular title, or by a particular author (e.g., "I am looking for a book on functions," or "I am looking for *The Theory of Functions* by Titchmarsh."). Given the subject, author, or title, one looks in the card catalog to find the *primary key*, the catalog number.

Similarly, we could build a catalog for our record collection consisting of entries for album title, composer, and artist. These fields are *secondary key fields*. Just as the library catalog relates an author entry (secondary key) to a card catalog number (primary key), so can we build an index file that relates Composer to Label ID, as illustrated in Fig. 6.6.

Along with the similarities, there is an important difference between this kind of secondary key index and the card catalog in a library. In a library, once you have the catalog number you can usually go directly to the

Composer index

Secondary key	*Primary key*
BEETHOVEN	ANG3795
BEETHOVEN	DG139201
BEETHOVEN	DG18807
BEETHOVEN	RCA2626
COREA	WAR23699
DVORAK	COL31809
PROKOFIEV	LON2312
RIMSKY-KORSAKOV	MER75016
SPRINGSTEEN	COL38358
SWEET HONEY IN THE R	FF245

FIGURE 6.6 Secondary key index organized by composer.

stacks to find the book since the books are arranged in order by catalog number. In other words, the books are sorted by primary key. The actual data records in our file, on the other hand, are *entry sequenced*. Consequently, after consulting the composer index to find the Label ID, you must consult one additional index, our primary key index, to find the actual byte offset of the record that has this particular Label ID. The procedure is summarized in Fig. 6.7.

Clearly it is possible to relate secondary key references (e.g., Beethoven) directly to a byte offset (211) rather than to a primary key (DG18807). However, there are excellent reasons for postponing this binding of a secondary key to a specific address for as long as possible. These reasons become clear as we discuss the way that fundamental file operations such as record deletion and updating are affected by the use of secondary indexes.

Record Addition When a secondary index is present, adding a record to the file means adding a record to the secondary index. The cost of doing this


```
PROCEDURE search_on_secondary(KEY)

    search for KEY in the secondary index

    once the correct secondary index record is found, set LABEL_ID
        to the primary key value in the record's reference field

    call retrieve_record(LABEL_ID) to get the data record

end PROCEDURE
```

FIGURE 6.7 *Search_on_secondary:* an algorithm to retrieve a single record from *Datafile* through a secondary key index.

is very similar to the cost of adding a record to the primary index: Either records must be shifted or a vector of pointers to structures needs to be rearranged. As with primary indexes, the cost of doing this decreases greatly if the secondary index can be read into electronic memory and changed there.

Note that the key field in the secondary index file is stored in canonical form (all of the composers' names are capitalized), since this is the form that we want to use when we are consulting the secondary index. If we want to print out the name in normal, mixed upper- and lowercase form, we can pick up that form from the original data file. Also note that the secondary keys are held to a fixed length, which means that sometimes they are truncated. The definition of the canonical form should take this length restriction into account if searching the index is to work properly.

One important difference between a secondary index and a primary index is that a secondary index can contain duplicate keys. In the sample index illustrated in Fig. 6.6, there are four records with the key BEETHOVEN. Duplicate keys are, of course, grouped together. Within this group, they should be ordered according to the values of the reference fields. In this example, that means placing them in order by Label ID. The reasons for this second level of ordering become clear a little later, as we discuss retrieval based on combinations of two or more secondary keys.

Record Deletion Deleting a record usually implies removing all references to that record in the file system. So, removing a record from the data file would mean removing not only the corresponding record in the primary index, but also all of the records in the secondary indexes that refer to this primary index record. The problem with this is that secondary indexes, like the primary index, are maintained in sorted order by key.

Consequently, deleting a record would involve rearranging the remaining records to close up the space left open by deletion.

This delete-all-references approach would indeed be advisable if the secondary index referenced the data file directly. If we did not delete the secondary key references, and if the secondary keys were associated with actual byte offsets in the data file, it could be difficult to tell when these references were no longer valid. This is another instance of the pinned-record problem. The reference fields associated with the secondary keys would be pointing to byte offsets that could, after deletion and subsequent space reuse in the data file, be associated with different data records.

But we have carefully avoided referencing actual addresses in the secondary key index. After we search to find the secondary key, we do another search, this time on primary key. Since the primary index *does* reflect changes due to record deletion, a search for the primary key of a record that has been deleted will fail, returning a record-not-found condition. In a sense, the updated primary key index acts as a kind of final check, protecting us from trying to retrieve records that no longer exist.

Consequently, one option that is open to us when we delete a record from the data file is to modify and rearrange only the primary key index. We could safely leave intact the references to the deleted record that exist in the secondary key indexes. Searches starting from a secondary key index that lead to a deleted record are caught when we consult the primary key index.

If there are a number of secondary key indexes, the savings that result from not having to rearrange all of these indexes when a record is deleted can be substantial. This is especially important when the secondary key indexes are kept on secondary storage. It is also important in an interactive system, where the user is waiting at a terminal for the deletion operation to complete.

There is, of course, a cost associated with this short cut: Deleted records take up space in the secondary index files. With a file system that undergoes few deletions, this is not usually a problem. With a somewhat more volatile file structure, it is possible to address the problem by periodically removing from the secondary index files all records that contain references that are no longer in the primary index. If a file system is so volatile that even periodic purging is not adequate, it is probably time to consider another index structure, such as a B-tree, which allows for deletion without having to rearrange a lot of records.

Record Updating In our discussion of record deletion, we find that the primary key index serves as a kind of protective buffer, insulating the secondary indexes from changes in the data file. This insulation extends to record updating as well. If our secondary indexes contain references directly to byte offsets in the data file, then updates to the data file that result in changing a record's physical location in the file also require updating the secondary indexes. But, since we are confining such detailed information to the primary index, data file updates affect the secondary index only when they change either the primary or the secondary key. There are three possible situations:

- *Update changes the secondary key:* If the secondary key is changed, then we may have to rearrange the secondary key index so it stays in sorted order. This can be a relatively expensive operation.
- *Update changes the primary key:* This kind of change has a large impact on the primary key index, but often requires only that we update the affected reference field (*Label_id* in our example) in all the secondary indexes. This involves searching the secondary indexes (on the unchanged secondary keys) and rewriting the affected fixed-length field. It does not require reordering of the secondary indexes unless the corresponding secondary key occurs more than once in the index. If a secondary key does occur more than once, there may be some local reordering, since records having the same secondary key are ordered by the reference field (primary key).
- *Update confined to other fields:* All updates that do not affect either the primary or secondary key fields do not affect the secondary key index, even if the update is substantial. Note that if there are several secondary key indexes associated with a file, updates to records often affect only a subset of the secondary indexes.

6.6 Retrieval Using Combinations of Secondary Keys

One of the most important applications of secondary keys involves using two or more of them in combination to retrieve special subsets of records from the data file. To provide an example of how this can be done, we will extract another secondary key index from our file of recordings. This one uses the recording's *title* as the key, as illustrated in Fig. 6.8. Now we can respond to requests such as

- Find the record with Label ID COL38358 (primary key access);
- Find all the recordings of Beethoven's work (secondary key—composer); and

Title index

Secondary key	*Primary key*
COQ D'OR SUITE	MER75016
GOOD NEWS	FF245
NEBRASKA	COL38358
QUARTET IN C SHARP M	RCA2626
ROMEO AND JULIET	LON2312
SYMPHONY NO. 9	ANG3795
SYMPHONY NO. 9	COL31809
SYMPHONY NO. 9	DG18807
TOUCHSTONE	WAR23699
VIOLIN CONCERTO	DG139201

FIGURE 6.8 Secondary key index organized by recording title.

- Find all the recordings titled "Violin Concerto" (secondary key—title).

What is more interesting, however, is that we can also respond to a request that *combines* retrieval on the composer index with retrieval on the title index, such as: Find all recordings of Beethoven's Symphony No. 9. Without the use of secondary indexes, this kind of request requires a sequential search through the entire file. Given a file containing thousands, or even just hundreds, of records, this is a very expensive process. But, with the aid of secondary indexes, responding to this request is simple and quick.

We begin by recognizing that this request can be rephrased as a *Boolean AND* operation, specifying the intersection of two subsets of the data file:

```
Find all data records with:
    composer = "BEETHOVEN" AND title = "SYMPHONY NO. 9"
```

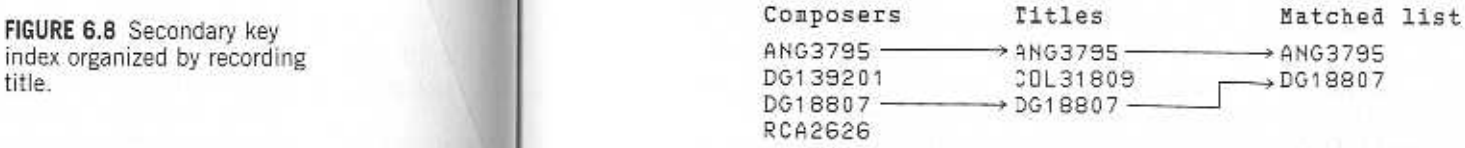

We begin our response to this request by searching the composer index for the list of Label IDs that identify records with Beethoven as the composer. (An exercise at the end of this chapter describes a binary search procedure that can be used for this kind of retrieval.) This yields the following list of Label IDs:

```
ANG3795
DG139201
DG18807
RCA2626
```

Next we search the title index for the Label IDs associated with records that have SYMPHONY NO. 9 as the title key:

```
ANG3795
COL31809
DG18807
```

Now we perform the Boolean AND, which is a match operation, combining the lists so only the members that appear in *both* lists are placed in the output list.



We give careful attention to algorithms for performing this kind of match operation in Chapter 7. Note that this kind of matching is much easier if the lists that are being combined are in sorted order. That is the reason why, when we have more than one entry for a given secondary key, the records are ordered by the primary key reference fields.

Finally, once we have the list of primary keys occurring in both lists, we can proceed to the primary key index to look up the addresses of the data file records. Then we can retrieve the records:

```
ANG ¦ 3795 ¦ Symphony No. 9 ¦ Beethoven ¦ Guilini
DG ¦ 18807 ¦ Symphony No. 9 ¦ Beethoven ¦ Karajan
```

This is the kind of operation that makes computer-indexed file systems useful in a way that far exceeds the capabilities of manual systems. We have only one copy of each data file record, and yet, working through the secondary indexes, we have multiple views of these records: We can look at them in order by title, by composer, or by any other field that interests us.

Using the computer's ability to combine sorted lists rapidly, we can even combine different views, retrieving *intersections* (Beethoven AND Symphony No. 9) or *unions* (Beethoven OR Prokofiev OR Symphony No. 9) of these views. And since our data file is entry sequenced, we can do all of this without having to sort data file records, confining our sorting to the smaller index records which can often be held in electronic memory.

Now that we have a general idea of the design and uses of secondary indexes, we can look at ways to improve these indexes so they take less space and require less sorting.

6.7 Improving the Secondary Index Structure: Inverted Lists

The secondary index structures that we have developed so far result in two distinct difficulties:

- We have to rearrange the index file *every time* a new record is added to the file, even if the new record is for an existing secondary key. For example, if we add another recording of Beethoven's Symphony No. 9 to our collection, both the composer and title indexes would have to be rearranged, even though both indexes already contain entries for secondary keys (but not the Label IDs) that are being added.
- If there are duplicate secondary keys, the secondary key field is repeated for each entry. This wastes space, making the files larger than necessary. Larger index files are less likely to be able to fit in electronic memory.

6.7.1 A First Attempt at a Solution

One simple response to these difficulties is to change the secondary index structure so it associates an *array* of references with each secondary key. For example, we might use a record structure that allows us to associate up to four Label ID reference fields with a single secondary key, as in

BEETHOVEN	ANG3795	DG139201	DG18807	RCA2626

Figure 6.9 provides a schematic example of how such an index would look if used with our sample data file.

Revised composer index

Secondary key	*Set of primary key references*			
BEETHOVEN	ANG3795	DG139201	DG18807	RCA2626
COREA	WAR23699			
DVORAK	COL31809			
PROKOFIEV	LON2312			
RIMSKY-KORSAKOV	MER75016			
SPRINGSTEEN	COL38358			
SWEET HONEY IN THE R	FF245			

FIGURE 6.9 Secondary key index containing space for multiple references for each secondary key.

The major contribution of this revised index structure is toward the solution of our first difficulty: the need to rearrange the secondary index file every time a new record is added to the data file. Looking at Fig. 6.9, we can see that the addition of another recording of a work by Prokofiev does not require the addition of another record to the index. For example, if we add the recording

```
ANG  36193  Piano Concertos 3 and 5  Prokofiev  Francois
```

we need to modify only the corresponding secondary index record by inserting a second Label ID:

PROKOFIEV	ANG36193	LON2312	

Since we are not adding another record to the secondary index, there is no need to rearrange any records. All that is required is a rearrangement of the fields in the existing record for Prokofiev.

Although this new structure helps avoid the need to rearrange the secondary index file so often, it does have some problems. For one thing, it provides space for only four Label IDs to be associated with a given key. In the very likely case that more than four Label IDs will go with some key, we need a mechanism for keeping track of the extra Label IDs.

A second problem has to do with space usage. Although the structure does help avoid the waste of space due to the repetition of identical keys, this space savings comes at a potentially high cost. By extending the fixed length of each of the secondary index records to hold more reference fields, we might easily lose more space to internal fragmentation than we gained by not repeating identical keys.

Since we don't want to waste any more space than we have to, we need to ask whether we can improve on this record structure. Ideally, what we would like to do is develop a new design, a revision of our revision, that

- Retains the attractive feature of not requiring reorganization of the secondary indexes for every new entry to the data file;
- Allows more than four Label IDs to be associated with each secondary key; and
- Does away with the waste of space due to internal fragmentation.

6.7.2 A Better Solution: Linking the List of References

Files such as our secondary indexes, in which a secondary key leads to a set of one or more primary keys, are called *inverted lists*. The sense in which the list is inverted should be clear if you consider that we are working our way backward from a secondary key to the primary key to the record itself.

The second word in the term inverted list also tells us something important: that we are, in fact, dealing with a *list* of primary key references. Our revised secondary index, which collects together a number of Label IDs for each secondary key, reflects this list aspect of the data more directly than did our initial secondary index. Another way of conceiving of this list aspect of our inverted list is illustrated in Fig. 6.10.

As Fig. 6.10 shows, an ideal situation would be to have each secondary key point to a different list of primary key references. Each of these lists could grow to be just as long as it needs to be. If we add the new Prokofiev record, the list of Prokofiev references becomes



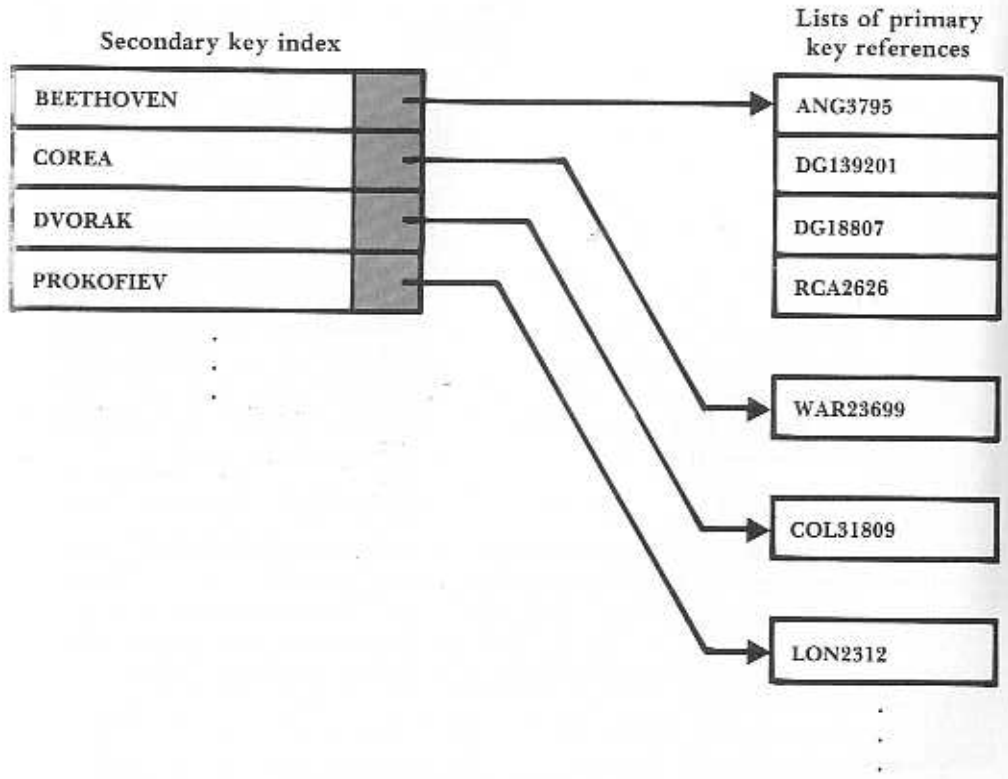


FIGURE 6.10 Conceptual view of the primary key reference fields as a series of lists.

Similarly, adding two new Beethoven recordings adds just two additional elements to the list of references associated with the Beethoven key. Unlike our record structure that allocates enough space for four Label IDs for each secondary key, the lists could contain hundreds of references, if needed, while still requiring only one instance of a secondary key. On the other hand, if a list requires only one element, then no space is lost to internal fragmentation. Most important of all, we need to rearrange only the file of secondary keys if a new composer is added to the file.

How can we set up an unbounded number of different lists, each of varying length, without creating a large number of small files? The simplest way is through the use of linked lists. We could redefine our secondary index so it consists of records with two fields—a secondary key field, and a field containing the relative record number of the first corresponding

primary key reference (Label ID) in the inverted list. The actual primary key references associated with each secondary key would be stored in a separate, entry-sequenced file.

Given the sample data we have been working with, this new design would result in a secondary key file for composers and an associated Label ID file that are organized as illustrated in Fig. 6.11. Following the links for the list of references associated with Beethoven helps us see how the Label ID List file is organized. We begin, of course, by searching the secondary key index of composers for Beethoven. The record that we find points us to relative record number (RRN) 3 in the Label ID List file. Since this is a fixed-length file, it is easy to jump to RRN 3 and read in its Label ID (ANG3795). Associated with this Label ID is a link to a record with RRN 8. We read in the Label ID for that record, adding it to our list (ANG379 DG139201). We continue following links and collecting Label IDs until the list looks like this:

```
ANG3795     DG139201     DG18807     RCA2626
```

The link field in the last record read from the Label ID List file contains a value of −1. As in our earlier programs, this indicates end-of-list, so we know that we now have all the Label ID references for Beethoven records.

To illustrate how record addition affects the Secondary Index and Label ID List files, we add the Prokofiev recording mentioned earlier:

```
ANG  36193  Piano Concertos 3 and 5  Prokofiev  Francois
```

You can see (Fig. 6.11) that the Label ID for this new recording is the last one in the Label ID List file, since this file is entry sequenced. Before this record is added, there is only one Prokofiev recording. It has a Label ID of LON2312. Since we want to keep the Label ID Lists in order by ASCII character values, the new recording is inserted in the list for Prokofiev so it logically precedes the LON2312 recording.

Associating the Secondary Index file with a new file containing linked lists of references provides some advantages over any of the structures considered up to this point:

- The only time we need to rearrange the Secondary Index file is when a new composer's name is added or an existing composer's name is changed (e.g., it was misspelled on input). Deleting or adding recordings for a composer who is already in the index involves changing only the Label ID List file. Deleting *all* the recordings for a composer could be handled by modifying the Label ID List file, while leaving the entry in the Secondary Index file in place, using a value of −1 in its reference field to indicate that the list of entries for this composer is empty.
- In the event that we do need to rearrange the Secondary Index file, the task is quicker now since there are fewer records and each record is smaller.
- Since there is less need for sorting, it follows that there is less of a penalty associated with keeping the Secondary Index files off on secondary storage, leaving more room in RAM for other data structures.
- The Label ID List file is entry sequenced. That means that it *never* needs to be sorted.
- Since the Label ID List file is a fixed-length record file, it would be very easy to implement a mechanism for reusing the space from deleted records, as described in Chapter 5.

Improved revision of the composer index

Secondary Index file

0	BEETHOVEN	3
1	COREA	2
2	DVORAK	7
3	PROKOFIEV	10
4	RIMSKY-KORSAKOV	6
5	SPRINGSTEEN	4
6	SWEET HONEY IN THE R	9

Label ID List file

0	LON2312	−1
1	RCA2626	−1
2	WAR23699	−1
3	ANG3795	8
4	COL38358	−1
5	DG18807	1
6	MER75016	−1
7	COL31809	−1
8	DG139201	5
9	FF245	−1
10	ANG36193	0

FIGURE 6.11 Secondary key index referencing linked lists of primary key references.

There is also at least one potentially significant disadvantage to this kind of file organization: The Label IDs associated with a given composer are no longer guaranteed to be physically grouped together. The technical term for such "togetherness" is *locality;* with a linked, entry-sequenced structure such as this, it is less likely that there will be locality associated with the logical groupings of reference fields for a given secondary key. Note, for example, that our list of Label IDs for Prokofiev consists of the very last and the very first records in the file. This lack of locality means that picking up the references for a composer that has a long list of references could involve a large amount of *seeking* back and forth on the disk. Note that this kind of seeking would not be required for our original Secondary Index file structure.

One obvious antidote to this seeking problem is to keep the Label ID List file in memory. This could be expensive and impractical, given many secondary indexes, except for the interesting possibility of using the same Label ID List file to hold the lists for a number of Secondary Index files. Even if the file of reference lists were too large to hold in memory, it might be possible to obtain a performance improvement by holding only a part of the file in memory at a time, paging sections of the file in and out of memory as they are needed.

Several exercises at the end of the chapter explore these possibilities more thoroughly. These are very important problems, since the notion of dividing the index into pages is fundamental to the design of B-trees and other methods for handling large indexes on secondary storage.

6.8 Selective Indexes

Another interesting feature of secondary indexes is that they can be used to divide a file into parts, providing a selective view. For example, it is possible to build a *selective index* that contains only the titles of classical recordings in the record collection. If we have additional information about the recordings in the data file, such as the date the recording was released, we could build selective indexes such as "recordings released prior to 1970" and "recordings since 1970." Such selective index information could be combined into Boolean AND operations to respond to requests such as, "List all the recordings of Beethoven's Ninth Symphony released since 1970." Selective indexes are sometimes useful when the contents of a file fall naturally and logically into several broad categories.

6.9 Binding

A recurrent and very important question that emerges in the design of file systems that utilize indexes is: *At what point in time is the key bound to the physical address of its associated record?*

In the file system we are designing in the course of this chapter, the *binding* of our primary keys to an address takes place *at the time the files are constructed.* The secondary keys, on the other hand, are bound to an address *at the time that they are actually used.*

Binding at the time of the file construction results in faster access. Once you have found the right index record, you have in hand the byte offset of the data record you are seeking. If we elected to bind our secondary keys to their associated records at the time of file construction, so when we find the DVORAK record in the composer index we would know immediately that the data record begins at byte 353 in the data file, secondary key retrieval would be simpler and faster. The improvement in performance is particularly noticeable if both the primary and secondary index files are used on secondary storage rather than in memory. Given the arrangement we designed, we would have to perform a binary search of the composer index and then a binary search of the primary key index before being able to jump to the data record. Binding early, at file construction time, does away entirely with the need to search on the primary key.

The disadvantage of binding directly in the file, of *binding tightly*, is that reorganizations of the data file must result in modifications to all bound index files. This reorganization cost can be very expensive, particularly with simple index files in which modification would often mean shifting records. By postponing binding until execution time, when the records are actually being used, we are able to develop a secondary key system that involves a minimal amount of reorganization when records are added or deleted.

Another important advantage of postponing binding until a record is actually retrieved is that this approach is safer. As we see in the system that we set up, associating the secondary keys with reference fields consisting of primary keys allows the primary key index to act as a kind of final check of whether a record is really in the file. The secondary indexes can afford to be wrong. This situation is very different if the secondary index keys are tightly bound, containing addresses. We would then be jumping directly from the secondary key into the data file; the address would need to be right.

This brings up a related safety aspect: It is always more desirable to have to make important changes in one place, rather than having to make them in many places. With a bind-at-retrieval-time scheme such as we developed, we need to remember to make a change in only one place, the primary key index, if we move a data record. With a more tightly bound system, we have to make many changes successfully to keep the system internally consistent, braving power failures, user interruptions, and so on.

When designing a new file system, it is better to deal with this question of binding *intentionally* and *early* in the design process, rather than letting the binding just happen. In general, tight, in-the-data binding is most attractive when

- The data file is static or nearly so, requiring little or no adding, deleting, and updating of records; and
- Rapid performance during actual retrieval is a high priority.

For example, tight binding is desirable for file organization on a mass-produced, read-only optical disk. The addresses will never change since no new records can ever be added; consequently, there is no reason not to obtain the extra performance associated with tight binding.

For file applications in which record addition, deletion, and updating do occur, however, binding at retrieval time is usually the more desirable option. Postponing binding for as long as possible usually makes these operations simpler and safer. If the file structures are carefully designed, and, in particular, if the indexes use more sophisticated organizations such as B-trees, retrieval performance is usually quite acceptable, even given the additional work required by a bind-at-retrieval system.

SUMMARY

We began this chapter with the assertion that indexing is an alternative to sorting as a way of structuring a file so records can be found *by key*. Unlike sorting, indexing permits us to perform *binary searches* for keys in variable-length record files. If the index can be held in memory, record addition, deletion, and retrieval can be done much more quickly with an indexed, entry-sequenced file than with a sorted file.

Indexes can do much more than merely improve on access time: They can provide us with new capabilities that are inconceivable with access methods based on sorted data records. The most exciting new capability

involves the use of multiple secondary indexes. Just as a library card catalog allows us to regard a collection of books in author order, title order, or subject order, so index files allow us to maintain different views of the records in a data file. We find that we cannot only use secondary indexes to obtain different views of the file, but that we can also combine the associated lists of primary key references and thereby combine particular views.

In this chapter we address the problem of how to rid our secondary indexes of two liabilities:

- The need to repeat duplicate secondary keys; and
- The need to rearrange the secondary indexes every time a record is added to the data file.

A first solution to these problems involves associating a fixed-size *vector* of reference fields with each secondary key. This solution results in an overly large amount of internal fragmentation but serves to illustrate the attractiveness of handling the reference fields associated with a particular secondary key as a group, or *list*.

Our next iteration of solutions to our secondary index problems is more successful and much more interesting. We can treat the primary key references themselves as an entry-sequenced file, forming the necessary lists through the use of *link fields* associated with each primary record entry. This allows us to create a secondary index file that, in the case of the composer index, needs rearrangement only when we add new composers to the data file. The entry-sequenced file of linked reference lists never requires sorting. We call this kind of secondary index structure an *inverted list*.

There are also, of course, disadvantages associated with our new solution. The most serious disadvantage is that our file demonstrates less locality: Lists of associated records are less likely to be physically adjacent. A good antidote to this problem is to hold the file of linked lists in memory. We note that this is made more plausible because a single file of primary references can link the lists for a number of secondary indexes.

As indicated by the length and breadth of our consideration of secondary indexing, multiple keys, and inverted lists, these topics are among the most interesting aspects of indexed access to files. The concepts of secondary indexes and inverted lists become even more powerful later, as we develop index structures that are themselves more powerful than the simple indexes that we consider here. But, even so, we already see that for small files consisting of no more than a few thousand records, approaches to inverted lists that rely merely on simple indexes can provide a user with a great deal of capability and flexibility.

KEY TERMS

Binding. Binding takes place when a key is associated with a particular physical record in the data file. In general, binding can take place either during the preparation of the data file and indexes or during program execution. In the former case, which is called *tight binding*, the indexes contain explicit references to the associated physical data record. In the latter case, the connection between a key and a particular physical record is postponed until the record is actually retrieved in the course of program execution.

Entry-sequenced file. A file in which the records occur in the order that they are entered into the file.

Index. An index is a tool for finding records in a file. It consists of a *key field* on which the index is searched and a *reference field* that tells where to find the data file record associated with a particular key.

Inverted list. The term *inverted list* refers to indexes in which a key may be associated with a *list* of reference fields pointing to documents that contain the key. The secondary indexes developed toward the end of this chapter are examples of inverted lists.

Key field. The key field is the portion of an index record that contains the canonical form of the key that is being sought.

Locality. Locality exists in a file when records that will be accessed in a given temporal sequence are found in physical proximity to each other on the disk. Increased locality usually results in better performance, since records that are in the same physical area can often be brought into memory with a single *read* request to the disk.

Reference field. The reference field is the portion of an index record that contains information about where to find the data record containing the information listed in the associated key field of the index.

Selective index. A selective index contains keys for only a portion of the records in the data file. Such an index provides the user with a view of a specific subset of the file's records.

Simple index. All the index structures discussed in this chapter are simple indexes insofar as they are all built around the idea of an ordered, linear sequence of index records. All these simple indexes share a common weakness: Adding records to the index is expensive. As we see later, tree-structured indexes provide an alternate, more efficient solution to this problem.

EXERCISES

1. Until now, it was not possible to perform a binary search on a variable-length record file. Why does indexing make binary search possible? With a fixed-length record file it *is* possible to perform a binary search. Does this mean that indexing need not be used with fixed-length record files?

2. Why is *title* not used as a primary key in the data file described in this chapter? If it were used as a secondary key, what problems would have to be considered in deciding on a canonical form for titles?

3. What is the purpose of keeping an out-of-date-status flag in the header record of an index? In a multiprogramming environment, this flag might be found to be set by one program because another program is in the process of reorganizing the index. How should the first program respond to this situation?

4. Explain how the use of an index pins the data records in a file.

5. When a record in a data file is updated, corresponding primary and secondary key indexes may or may not have to be altered, depending on whether the file has fixed- or variable-length records, and depending on the type of change made to the data record. Make a list of the different updating situations that can occur, and explain how each affects the indexes.

6. Discuss the problem that occurs when you add the following recording to the recordings file, assuming that the composer index shown in Fig. 6.9 is used. How might you solve the problem without substantially changing the secondary key index structure?

```
LON  1259  Fidelio  Beethoven  Maazel
```

7. What is an inverted list, and when is it useful?

8. How are the structures in Fig. 6.11 changed by the addition of the recording

```
LON  1259  Fidelio  Beethoven  Maazel
```

9. Suppose you have the data file described in this chapter, greatly expanded, with a primary key index and secondary key indexes organized by composer, artist, and title. Suppose that an inverted list structure is used to organize the secondary key indexes. Give step-by-step descriptions of how a program might answer the following queries:
   a. List all recordings of Bach or Beethoven; and
   b. List all recordings by Perleman of pieces by Mozart or Joplin.

10. One possible antidote to the problem of diminished locality when using inverted lists is to use the same Label ID List file to hold the lists for several of the secondary index files. This increases the likelihood that the secondary indexes can be kept in primary memory. Draw a diagram of a single Label ID List file that can be used to hold references for both the secondary index of composers and the secondary index of titles. How would you handle the difficulties that this arrangement presents with regard to maintaining the Label ID List file?

11. Discuss the following structures as antidotes to the possible loss of locality in a secondary key index:

- Leave space for multiple references for each secondary key (Fig. 6.9).
- Allocate variable-length records for each secondary key value, where each record contains the secondary key value, followed by the Label IDs, followed by free space for later addition of new Label IDs. The amount of free space left could be fixed, or it could be a function of the size of the original list of Label IDs.

12. The method and timing of binding affect two important attributes of a file system—speed and flexibility. Discuss the relevance of these attributes, and the effect of binding time on them, for a hospital patient information system designed to provide information about current patients by patient name, patient ID, location, medication, doctor or doctors, and illness.

Programming and Design Exercises

13. Implement the *retrieve_record( )* procedure outlined in Fig. 6.4.

14. In solving the preceding problem, you have to create a mechanism for deciding how many bytes to read from the *Datafile* for each record. At least four options are open to you:

a. Jump to the *byte_offset*, read the size field, then use this information to read the record.

b. Build an index file that contains a record size field that reflects the *true* size of the data record, including the size field carried in the *Datafile*. Use the size field carried in the index file to decide how many bytes to read.

c. Follow much the same strategy as in option (b), except use a *Datafile* that does not contain internal size fields.

d. Jump to the *byte_offset* and read a fixed, overly large number of bytes (e.g., 512 bytes). Once these bytes are read into a memory buffer, use the size field at the start of the buffer to decide how many bytes to break out of the buffer.

Evaluate each of these options, listing the advantages and disadvantages of each.

15. Implement procedures to read in and to write back the *INDEX[ ]* array to the index file.

16. When searching secondary indexes that contain multiple records for some of the keys, we do not want to find just *any* record for a given secondary key; we want to find the *first* record containing that key. Finding the first record allows us to read ahead, sequentially, extracting all of the records for the given key. Write a variation of a binary search function that returns the relative record number of the *first* record containing the given key. The function should return a negative value if the key cannot be found.

17. If a Label ID List file such as the one shown in Fig. 6.11 is too large to be held in memory in its entirety, it might still be possible to improve performance by retaining a number of blocks of the file in memory. These blocks are called *pages*. Since the records in the Label ID List file are each 16 bytes long, a page might consist of 32 records (512 bytes). Write a function that would hold the most recently used eight pages in memory. Calls for a specific record from the Label ID List file would be routed through this function. It would check to see if the record exists in one of the pages that is already in memory. If so, the function would return the values of the record fields immediately. If not, the function would read in the page containing the desired record, either writing out or dumping the page that was used least recently. Clearly, if a page has been changed, it needs to be written out rather than dumped. When the program is over, all pages still in memory must be checked to see if they should be written out.

18. Assuming the use of a paged index as described in the preceding problem, and given that the Label ID List file is entry sequenced, is there any particular order of data entry (initial file loading) that tends to give better performance than other methods? How does the use of an organization method such as that described in problem 10, which combines the linked lists from several secondary indexes into a single file, affect your answer about performance?

19. The Label ID List file is entry sequenced. Development of paging schemes is simpler for entry-sequenced files than for files that are kept in sorted order. List the additional difficulties involved in the design of a paging system for a sorted index, such as the primary key index. Accepting the possibility that there will be a number of pages that are only partially full, design such a paging system. How will you handle the insertion of a new key when the page in which it belongs is full?

FURTHER READINGS

We have much more to say about indexing in later chapters, where we take up the subjects of tree-structured indexes and of indexed sequential file organizations. The topics developed in the current chapter, particularly those relating to secondary indexes and inverted files, are also covered by many other file and data structure texts. The few texts that we list here are of interest because they either develop certain topics in more detail or present the material from a different viewpoint.

Wiederhold (1983) provides a survey of many of the index structures we discuss, along with a number of others. His treatment is more mathematical than that provided in our text. Users interested in looking at indexed files in the context of PL/I and of large IBM mainframes will want to see Bradley (1982). A brief, readable overview of a number of different file organizations is provided in J. D. Ullman (1980).

Tremblay and Sorenson (1984) provide a comparison of inverted list structures with an alternative organization called *multilist* files. M. E. S. Loomis (1983) provides a similar discussion, along with some examples oriented toward COBOL users. Salton and McGill (1983) discuss inverted lists in the context of their application in information retrieval systems.

7 Cosequential Processing and the Sorting of Large Files

CHAPTER OBJECTIVES

- Describe a class of frequently used processing activities known as *cosequential processes*.
- Provide a general model for implementing all varieties of cosequential processes.
- Illustrate the use of the model to solve a number of different kinds of cosequential processing problems, including problems other than simple merges and matches.
- Introduce *heapsort* as an approach to overlapping I/O with sorting in RAM.
- Show how merging provides the basis for sorting very large files.
- Examine the costs of K-way merges on disk and find ways to reduce those costs.
- Introduce the notion of *replacement selection*.
- Examine some of the fundamental concerns associated with sorting large files using tapes rather than disks.
- Introduce UNIX utilities for sorting, merging, and cosequential processing.